

CS339: Abstractions and Paradigms for Programming

OO: Visitor Pattern

Manas Thakur
CSE, IIT Bombay



Autumn 2025

Recap: Java Primer

- Variables and methods can be defined only inside classes
- Classes can be instantiated to create objects
- A class can extend another class and inherit its fields
- A class can extend another class and override its methods
- Parent class references can point to child class objects, but not vice versa
- A class can extend only one class but implement any number of interfaces
- All methods are virtual by default — resolved based on the “receiver” object
- Java class libraries provide useful data-structure implementations such as Vector, ArrayList, HashSet, HashMap, etc.



Let's look at an OO problem

- **Goal:** We want to sum the elements of an integer linked list:

```
interface List {}  
  
class Nil implements List {}  
  
class Cons implements List {  
    int head;  
    List tail;  
}
```



Approach 1: Instance checks and Typecasts

```
List l; ...
int sum = 0;
boolean proceed = true;
while (proceed) {
    if (l instanceof Nil) {
        proceed = false;
    } else if (l instanceof Cons) {
        sum = sum + ((Cons) l).head;
        l = ((Cons) l).tail;
    }
}
```

- **Bad:** Typecasts and instanceof checks are ugly (and expensive).
- **Good:** No need to touch Nil and Cons (abstraction principles respected).



Approach 2: Utilize OO style

```
interface List {  
    int sum();  
}  
class Nil implements List {  
    public int sum() { return 0; }  
}  
class Cons implements List {  
    int head;  
    List tail;  
    public int sum() {  
        return head + tail.sum();  
    }  
}
```

- **Good:** No typecasts and instanceofs.
- **Bad:** Original classes to be recompiled for each new operation.



Approach 3: Visitor Pattern

- Divide code into an object structure and a **Visitor**.
- Insert an `accept` method in each structure class. Each `accept` method takes a `Visitor` as an argument.
- A `Visitor` contains a `visit` method for each structure class.

```
interface List {  
    void accept(Visitor v);  
}  
interface Visitor {  
    void visit(Nil x);  
    void visit(Cons x);  
}
```



Approach 3: Visitor Pattern

- The purpose of accept methods is to invoke the visit method in the Visitor that can handle the current object.

```
class Nil implements List {  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}  
  
class Cons implements List {  
    int head;  
    List tail;  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}
```



Approach 3: Visitor Pattern

- The control flows back and forth between the `visit` methods in the `Visitor` and the `accept` methods in the object structure.

```
class SumVisitor implements Visitor {  
    int sum = 0;  
    public void visit(Nil x) {}  
    public void visit(Cons x) {  
        sum = sum + x.head;  
        x.tail.accept(this);  
    }  
}  
List l;  
...  
SumVisitor sv = new SumVisitor();  
l.accept(sv);  
System.out.println(sv.sum);
```



Visitor Pattern: Everything together

```
interface List {  
    void accept(Visitor v);  
}  
interface Visitor {  
    void visit(Nil x);  
    void visit(Cons x);  
}
```

```
class SumVisitor implements Visitor {  
    int sum = 0;  
    public void visit(Nil x) {}  
    public void visit(Cons x) {  
        sum = sum + x.head;  
        x.tail.accept(this);  
    }  
}  
List l;  
...  
SumVisitor sv = new SumVisitor();  
l.accept(sv);  
System.out.println(sv.sum);
```

```
class Nil implements List {  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}  
class Cons implements List {  
    int head;  
    List tail;  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}
```

Several compilers use
visitor pattern to perform
syntax-directed translation.



For some other day

- There are several more interesting OO patterns that help solve problems elegantly
- Particularly notable ones (find out as a **homework**):
 - *Strategy Pattern*
 - *Decorator Pattern*
 - *Factory-method Pattern*
- Learn more (not in this course):

